

EnterPrise GPT (Phase 1 VS Phase 2)

AI Enterprise GPT Backend — Phase 1 vs Phase 2

Repository:

ai-enterprise-gpt-backend

Phase 1 Branch:

main

Phase 2 Branch:

dev

Table of Contents

1.

Overview

2.

Phase 1 — Main Branch

◦

Architecture

◦

Project Structure

◦

Core Modules & Features

◦

API Endpoints

◦

Database Models

◦

Dependencies

3.

Phase 2 — Dev Branch

◦

Architecture Changes

◦

New Modules

◦

Enhanced Modules

◦

New API Endpoints

◦

Database Schema Changes

◦

New Dependencies

4.

Side-by-Side Feature Comparison

5.

Migration Notes

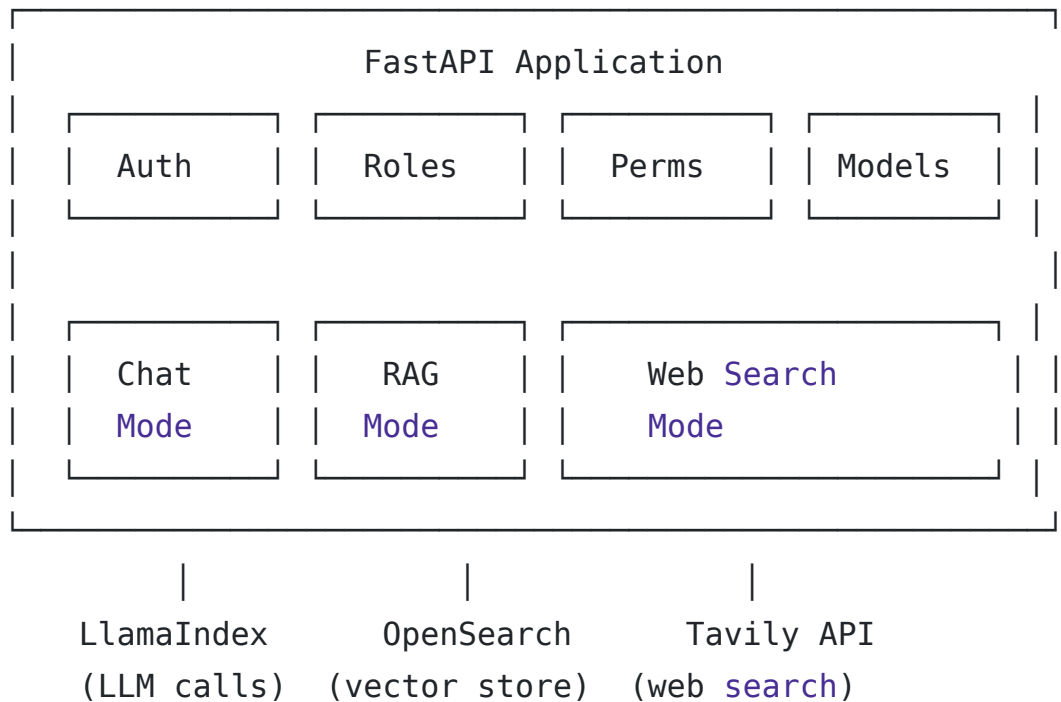
1. Overview

Attribute	Phase 1 (main)	Phase 2 (dev)
-----------	------------------	-----------------

Stage	Production-stable	Active development
AI Mode	LLM Chat + RAG + Web Search (separate)	Unified Orchestration Engine (all modes merged)
Workspace	Per-user flat conversations	Project-scoped workspaces with memory
Agents	None	ReAct Agent + LangGraph multi-tool
Storage	Flat document list	Folder-organised documents
LLM Framework	LlamaIndex only	LlamaIndex + LangChain/LangGraph
Database Migrations	Manual / Alembic baseline	Full Alembic version chain (5 migrations)
Test Coverage	Basic auth + model tests	Comprehensive unit + integration suites

2. Phase 1 — Main Branch

2.1 Architecture



All three AI modes (Chat, RAG, Web Search) are **independent, separate routers** with no shared orchestration layer.

2.2 Project Structure

```

ai-enterprise-gpt-backend/
├── app/
│   ├── ai/
│   │   ├── embedding/           # HuggingFace embedding component
│   │   ├── ingest/             # Document ingestion service
│   │   ├── llm/
│   │   │   ├── api/            # API LLM providers (OpenAI,
Anthropic, Gemini, Groq, Grok)
│   │   │   └── cloud/          # Cloud providers (AWS, Azure,
GCP)
│   │   │       ├── local/      # LlamaCPP local model loader
│   │   │       ├── factory.py  # LLM factory
│   │   │       ├── services.py # get_llm_from_db_model()
│   │   │       └── utils/      # prompt_helper, utils
│   │   ├── memory/            # (placeholder)
│   │   └── models/            # Model CRUD (domain, routes,
schemas, services)
  
```

```

|   |   |   └─ modes/
|   |   |   └─ chat_mode/           # LLM streaming chat
|   |   |   └─ rag_mode/           # Document RAG chat
|   |   |   └─ web_search_mode/    # Tavily web search + LLM
synthesis
|   |   |   └─ rag/                 # (placeholder)
|   |   |   └─ services/           # base_services
|   |   |   └─ vector_store/       # OpenSearch vector store
|   |   └─ apis/
|   |   └─ auth/                   # JWT auth, registration,
password, users
|   |   |   └─ permissions/        # Permission CRUD
|   |   |   └─ roles/              # Role CRUD + RBAC dependencies
|   |   └─ core/
|   |   |   └─ cache.py
|   |   |   └─ common_schemas.py
|   |   |   └─ constants.py
|   |   |   └─ di.py               # Injector-based DI
|   |   |   └─ encoders.py
|   |   |   └─ minio.py
|   |   |   └─ paths.py
|   |   |   └─ rbac_middleware.py
|   |   |   └─ settings.py
|   |   |   └─ yaml.py
|   |   └─ database.py
|   |   └─ fastapi_app.py          # App factory, middleware, route
registration
|   |   └─ init_db.py
|   |   └─ main.py
└─ config/settings.yaml
└─ requirements.txt
└─ Dockerfile
└─ docker-compose.yml
└─ Jenkinsfile

```

2.3 Core Modules & Features

Authentication & RBAC (`app/apis/`)

- **JWT Auth** — Login, logout (blacklist), token refresh
- **Email Verification** — OTP-based verification on registration
- **Password Management** — Forgot/reset with OTP, change password
- **RBAC** — Role-based access; 13 permissions across 4 resource types
- **User Management** — Admin CRUD, role assignment; protects default admin from deletion/modification
- **Role Management** — Full CRUD; auto-assigns default permissions on role creation

LLM Chat ([app/ai/modes/chat_mode/](#))

- **Streaming** via `StreamingResponse` (text/event-stream)
- **Multi-provider** — OpenAI, Anthropic, Gemini, Groq, Grok, AWS Bedrock, Azure OpenAI, GCP Vertex AI, Local LlamaCPP
- **Conversation history** — Persisted `Conversation` + `Message` DB rows
- **RAG integration** — `is_rag` flag + `document_ids` passed to same `/completions` endpoint
- **Source citation** — RAG context stored as JSON `sources` on message rows
- **Model tracking** — `model_id` stored per message; resolved to `model_name` on retrieval
- **Title** — First 50 chars of user prompt becomes conversation title

RAG Mode ([app/ai/modes/rag_mode/](#))

- **Document ingestion** — PDF, DOCX, DOC, TXT; file extension + duplicate name check
- **Vector store** — OpenSearch via LlamaIndex
- **Per-user isolation** — `user_id` scoped index filtering
- **RAG chat** — Non-streaming `/completions` , returns `RagChatResponse`
- **Conversations** — Separate RAG conversation + message tables
- **Documents** — List, delete endpoints

Web Search Mode ([app/ai/modes/web_search_mode/](#))

- **Tavily Search API** integration
- **LLM synthesis** — Streaming LLM-synthesized answers from search results
- **Separate router** — `/api/v1/web-search/completions`
- **Source citations** — Web search snippets stored as `sources`

 Model Management ([app/ai/models/](#))

- **12 provider types** — Local, API, Cloud variants
- **Full CRUD** — Create, list, get, update, delete model configs
- **HuggingFace import** — Background GGUF download to MinIO
- **Model testing** — Live connection validation
- **Model fetching** — Hardcoded allowlists for each provider's available models
- **Usage tracking** — Token counts, cost, response times

 Infrastructure

- **PostgreSQL 16** — Primary relational DB (SQLAlchemy 2.0)
- **OpenSearch 2.12** — Vector store
- **MinIO** — S3-compatible object storage for model files
- **Docker Compose** — Full orchestration stack
- **Jenkins CI/CD** — SSH-based deployment with health checks
- **Alembic** — Database migrations (baseline migration only)

2.4 API Endpoints

Prefix	Tag	Key Endpoints
/auth	Authentication	POST /login , POST /logout , POST /register , POST /refresh
/email	Email	POST /verify , POST /resend
/password	Password	POST /forgot , POST /reset , POST /change
/users	User Management	GET / , POST / , GET /{id} , PUT /{id} , DELETE /{id}
/roles	Role Management	GET / , POST / , PUT /{id} , DELETE /{id}
/role_permissions	Role Permissions	GET / , POST / , DELETE /

/permissions	Permissions	GET /
/models	Model Management	GET / , POST / , GET /{id} , PUT /{id} , DELETE /{id} , POST /{id}/test , GET /fetch/{provider}
/api/v1/llm	LLM Chat	POST /completions , GET /conversations , GET /conversations/{id}/messages , POST /conversations , DELETE /conversations/{id} PUT /conversations/{id}
/api/v1/rag	RAG	POST / (ingest), GET /documents , DELETE /documents/{id} , POST /completions , GET /conversations , POST /conversations , GET /conversations/{id}/messages , DELETE /conversations/{id}
/api/v1/web-search	Web Search	POST /completions , GET /health

2.5 Database Models

Tables:

Table	Description
-------	-------------

users	User accounts with email, password hash, role
roles	Named roles (e.g., Admin, User)
permissions	Resource-action permission definitions
role_permissions	Many-to-many role ↔ permission mapping
ai_models	LLM model configurations (all provider types)
chat_conversations	Chat conversation sessions
chat_messages	Individual messages (role, content, sources JSON)
rag_conversations	Separate RAG conversation sessions
rag_messages	RAG conversation messages
token_blacklist	Revoked JWT tokens
email_otps	OTP codes for email verification / password reset

chat_messages columns (Phase 1):

id, conversation_id, model_id, role, content, sources (JSON), created_at

2.6 Dependencies

AI/ML Stack:

- llama-index-core>=0.14.0
- llama-index-llms-openai
- llama-index-llms-anthropic
- llama-index-llms-gemini
- llama-index-llms-llama-cpp
- llama-index-embeddings-huggingface
- llama-index-vector-stores-opensearch

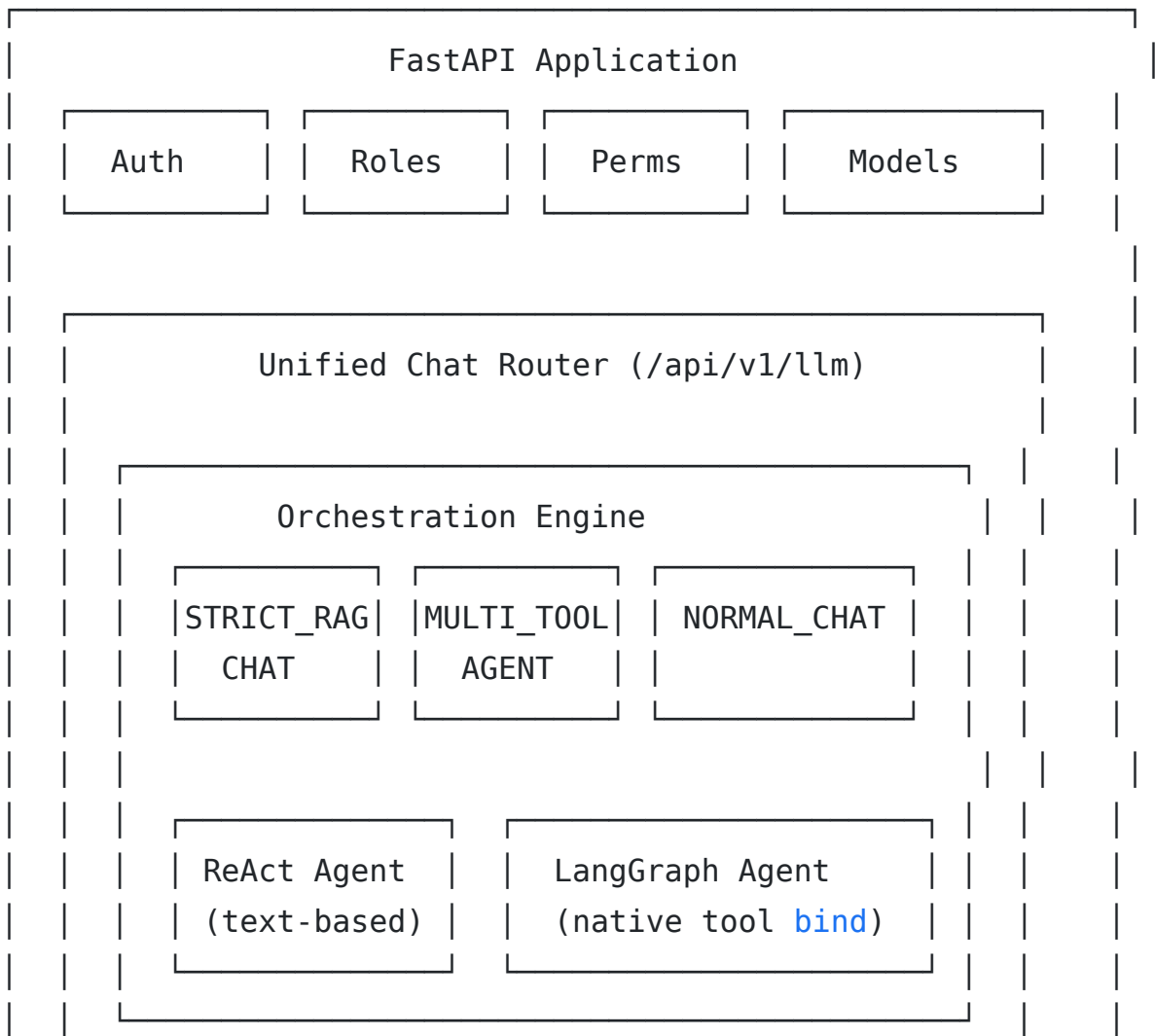
```
llama-index-readers-file
llama-cpp-python
openai>=1.0.0
anthropic>=0.7.0
groq>=0.4.0
xai-sdk
google-genai==1.56.0
tavily-python>=0.5.0
```

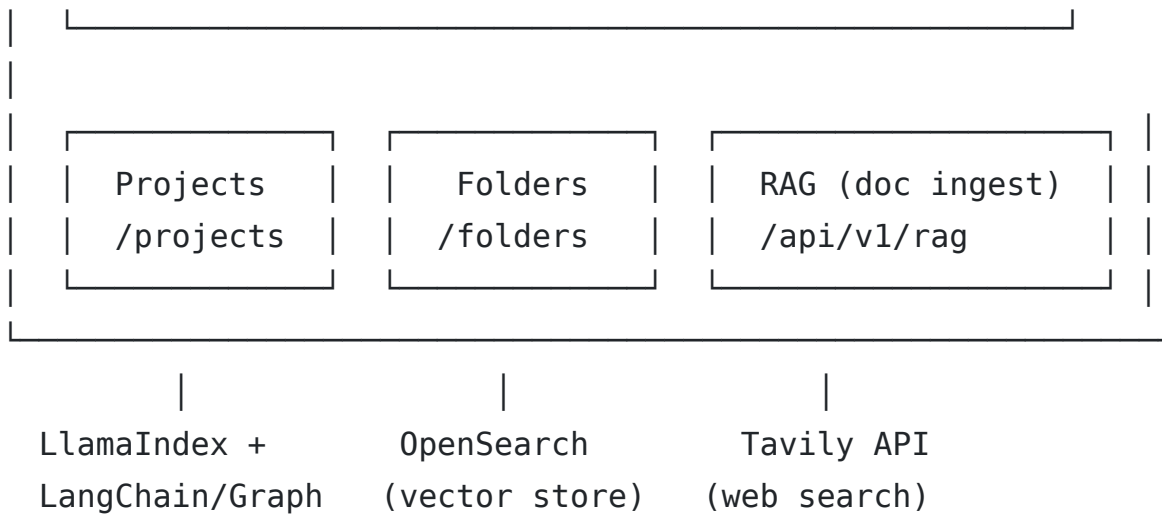
No LangChain in Phase 1.

3. Phase 2 — Dev Branch

3.1 Architecture Changes

Phase 2 introduces a **Unified Orchestration Engine** that replaces the three separate AI modes:





Key architectural decision: Web Search mode is no longer a separate router. It becomes a **tool** (`web_search`) inside the orchestration engine alongside `document_search` .

3.2 New Modules

`app/ai/orchestration/` — Chat Orchestration Engine

The core of Phase 2. Replaces the fragmented chat/rag/web-search services.

File	Purpose
<code>chat_chain.py</code>	Main orchestrator <code>run_chat_orchestrator()</code> — 436 lines. Routes to STRICT_RAG_CHAT, MULTI_TOOL_AGENT, or NORMAL_CHAT based on request params. Handles streaming SSE output, source attribution, project memory injection.
<code>text_react_agent.py</code>	Text-based ReAct loop fallback (168 lines) for LLMs that don't support native tool binding (e.g., local models). Parses Thought/Action/Observation/Final Answer format manually.

<code>router.py</code>	Thin router that delegates to orchestration engine
------------------------	--

Route Types (decided by `run_chat_orchestrator`):

Route Type	Trigger	Behaviour
<code>STRICT_RAG_CHAT</code>	<code>is_rag=True</code> or <code>document_ids</code> provided	Structured "Verification AI" prompt with 3 context tiers [SELECTED], [ALL_DOCS], [LLM]. Tags source in stream.
<code>MULTI_TOOL_AGENT</code>	<code>selected_tools</code> contains <code>web_search</code>	LangGraph agent with <code>document_search</code> + <code>web_search</code> tools. Falls back to text ReAct for unsupported models.
<code>NORMAL_CHAT</code>	Default	Plain LLM streaming with project memory + conversation history

`app/ai/projects/` — Project Workspaces

Agentic workspaces that scope conversations and documents.

File	Purpose
<code>models.py</code>	<code>Project</code> SQLAlchemy model (<code>ai_projects</code> table)
<code>schemas.py</code>	<code>ProjectCreate</code> , <code>ProjectUpdate</code> , <code>ProjectResponse</code> Pydantic schemas
<code>routes.py</code>	Full CRUD + validation (<code>GET /</code> , <code>POST /</code> , <code>PUT /{id}</code> , <code>DELETE /{id}</code> , <code>GET /{id}</code>)
<code>memory_services.py</code>	Background <code>aggregate_project_memory_backg</code>

01/07/2026, 15:31	EnterPrise GPT (Phase 1 VS Phase 2)	<code>round()</code> — summarises project conversations into <code>memory_summary</code> using Conversation Summary Buffer logic
-------------------	-------------------------------------	--

Project Model fields:

```
id, user_id, name, description, system_prompt,
memory_summary (compressed long-term memory),
created_at, updated_at
```

Memory logic: After 4 Q&A pairs (8 messages), the oldest 6 are summarised via LLM into `memory_summary` . The most recent 2 are kept raw for active context.

app/ai/folders/ — Document Folders

Organises user documents into named folders.

File	Purpose
<code>models.py</code>	<code>DocumentFolder</code> SQLAlchemy model (<code>document_folders</code> table)
<code>schemas.py</code>	<code>FolderCreate</code> , <code>FolderResponse</code> Pydantic schemas
<code>routes.py</code>	Full CRUD — <code>GET /</code> , <code>POST /</code> , <code>PUT /{id}</code> , <code>DELETE /{id}</code> with alphabetic name validation

app/ai/tools/ — Agent Tool Registry

File	Purpose
<code>registry.py</code>	<code>get_agent_tools()</code> — returns list of LangChain tools based on user selection. Defines <code>document_search_tool</code> (calls RAG service) and <code>web_search_tool</code> (calls Tavily).

app/ai/llm/utils/langchain_adapter.py — LlamaIndex → LangChain Bridge

`LlamaIndexChatAdapter(BaseChatModel)` wraps any LlamaIndex LLM to make it compatible with LangChain/LangGraph interfaces. Implements `_generate`, `_agenerate`, and `_astream` with message conversion.

3.3 Enhanced Modules

Chat Mode (`app/ai/modes/chat_mode/`)

`chat_router.py` changes:

- `POST /completions` now accepts `project_id` and `selected_tools` in addition to `document_ids`, `is_rag`
- Delegates to `run_chat_orchestrator()` instead of `async_chat_completions()`
- `GET /conversations` now filters by `project_id` query param
- Returns SSE headers (`X-Accel-Buffering: no`, `Cache-Control: no-cache`)

`chat_schemas.py` changes:

- `ChatRequest` gains `project_id: Optional[int]` and `selected_tools: Optional[List[str]]`
- `ConversationResponse` gains `project_id`, `chat_mode`, `memory_scope` fields

`models.py` — `Conversation` gains:

```
project_id = Column(Integer, ForeignKey("ai_projects.id",
ondelete="SET NULL"))
chat_mode   = Column(String(50), default="auto")
memory_scope = Column(String(50), default="conversation")
```

`models.py` — `Message` gains:

```
tool_calls    = Column(JSON)           # Records tool execution details
route_type    = Column(String(50))     # STRICT_RAG_CHAT |
MULTI_TOOL_AGENT | NORMAL_CHAT
```

```
is_summarized = Column(Boolean, default=False) # Folded into
project memory?
```

New DocumentTask model:

```
# Tracks background document ingestion status
id, user_id, folder_id, file_name, file_size, status, message,
created_at
```

RAG Mode (`app/ai/modes/rag_mode/`)

`rag_router.py` changes:

- Ingest endpoint accepts `folder_id: Optional[int]` (Form field)
- Duplicate detection upgraded: **SHA-256 content hash** check in addition to filename check
- **Storage limit** enforcement based on `settings.rag.max_upload_size_mb`
- Document ingestion now runs in **background task** (non-blocking)
- `DocumentResponse` gains `folder_id` and `message` fields

`rag_services.py` changes:

- `search_categorized_context()` — New method that returns `{selected_context, other_context}` split (for STRICT_RAG_CHAT tier system)
- Message history uses **sliding window** (limited recent messages, not all history)

LLM Services (`app/ai/llm/services.py`)

- Added **Grok provider** support
- LlamaIndex adapter integration
- Model download progress tracking improvements

FastAPI App (`app/fastapi_app.py`)

Phase 2 registers additional routers:

Removed

```
from app.ai.modes.web_search_mode.web_search_router import  
web_search_router # ❌ removed
```

Added

```
from app.ai.projects.routes import project_router # ✅ new  
from app.ai.folders.routes import folder_router # ✅ new
```

New route prefixes registered:

```
/api/v1/projects → project_router  
/api/v1/folders → folder_router
```

3.4 New API Endpoints

Projects API — **/api/v1/projects**

Method	Path	Description
GET	/	List all projects for current user
POST	/	Create new project (name, description, system_prompt)
GET	/ {project_id}	Get a specific project
PUT	/ {project_id}	Update project details
DELETE	/ {project_id}	Delete project + cascade conversations

Validation rules:

- Name: alphabets + spaces only, required, at most 255 chars
- Description: max 500 chars
- System Prompt (instructions): max 2000 chars

Folders API — `/api/v1/folders`

Method	Path	Description
GET	<code>/</code>	List all folders for current user
POST	<code>/</code>	Create folder (alphabets + spaces, max 255 chars, no duplicates)
GET	<code>/ {folder_id}</code>	Get folder + its documents
PUT	<code>/ {folder_id}</code>	Rename folder
DELETE	<code>/ {folder_id}</code>	Delete folder (documents unlinked or deleted)

Enhanced Chat Endpoint — `/api/v1/llm/completions`

New request body fields:

```
{
  "user_prompt": "string",
  "conversation_id": 123,
  "project_id": 5,
  "document_ids": ["doc1", "doc2"],
  "selected_tools": ["document_search", "web_search"],
  "is_rag": false
}
```

Enhanced Conversations — `/api/v1/llm/conversations`

- `GET /?project_id=5` — Filter conversations by project

3.5 Database Schema Changes

New Tables (Phase 2):

Table	Description
-------	-------------

ai_projects	Project workspaces (id , user_id , name , description , system_prompt , memory_summary)
document_folders	Document folder organisation (id , user_id , name , created_at)
document_tasks	Background ingestion task tracking (id , user_id , folder_id , file_name , file_size , status , message)

Altered Tables (Phase 2):

Table	New Columns
chat_conversations	project_id (FK → ai_projects), chat_mode , memory_scope
chat_messages	tool_calls (JSON), route_type (String), is_summarized (Boolean)
document_tasks	folder_id (FK → document_folders)

Alembic Migration Chain (Phase 2):

20260422_0000_baseline.py	
└─ 20260424_1140_add_project_tables_and_conversation_fk.py	
└─ 20260425_1725_add_is_summarized_to_message.py	
└─	
20260502_1111_unify_document_management_and_remove_.py	
└─	
20260502_1430_add_document_folders.py	

3.6 New Dependencies

Phase 2 adds **LangChain / LangGraph** stack on top of Phase 1's LlamaIndex stack:

LangChain Orchestration & Agents

langchain>=0.1.0

langgraph>=0.0.20

langchain-openai

langchain-anthropic

langchain-tavily

duckduckgo-search>=5.0.0

4. Side-by-Side Feature Comparison

Feature	Phase 1 (main)	Phase 2 (dev)
Chat Routing	Manual is_rag flag	Auto-routing: STRICT_RAG / MULTI_TOOL / NORMAL
RAG Tiers	Single context pool	3-tier: [SELECTED] → [ALL_DOCS] → [LLM]
Web Search	Separate router	Tool inside MULTI_TOOL_AGENT
AI Agents	✗ None	✓ LangGraph + text ReAct fallback
Project Workspaces	✗ None	✓ Full CRUD, scoped conversations
Long-term Memory	✗ None	✓ Conversation Summary Buffer per project
Document Folders	✗ Flat list	✓ Folder-organised with CRUD
Document Dedup	Filename only	Filename + SHA-256 hash
Storage Limit	✗ None	✓ Configurable max_upload_size_mb
Background Ingest	Blocking (sync)	✓ BackgroundTasks (non blocking)

Conversation Scope	Global per user	Project-scoped or global
Message Metadata	<code>sources</code> , <code>model_id</code>	+ <code>tool_calls</code> , <code>route_type</code> , <code>is_summarized</code>
LLM Framework	LlamaIndex only	LlamaIndex + LangChain + LangGraph
LlamaIndex Adapter	✗	✓ <code>LlamaIndexChatAdapter(BaseChatModel)</code>
Source Attribution	Post-hoc JSON chunk	Real-time tagged streaming per tier
SSE Headers	None	<code>X-Accel-Buffering: no</code> , <code>Cache-Control: no-cache</code>
Alembic Migrations	Baseline only	5-version migration chain
Test Coverage	Auth + model tests	+ Unit tests for all AI services, orchestration, agents
Message Sliding Window	Last 10 messages	Configurable sliding window

5. Migration Notes

Database

When moving from Phase 1 (`main`) to Phase 2 (`dev`) run Alembic upgrades in order:

```
alembic upgrade head
```

This will apply all 5 migrations including:

- Creating `ai_projects` and `document_folders` tables

- Adding `project_id` , `chat_mode` , `memory_scope` to `chat_conversations`
- Adding `tool_calls` , `route_type` , `is_summarized` to `chat_messages`
- Creating `document_tasks` table
- Adding `folder_id` FK to `document_tasks`

Environment Variables

Phase 2 requires additional keys for LangChain tools:

```
# Existing (Phase 1)
TAVILY_API_KEY=...
```

```
# New (Phase 2) – same key, now used by langchain-tavily
TAVILY_API_KEY=...
```

No new env vars required; same keys are reused by LangChain wrappers.

Web Search

Phase 1 clients calling `POST /api/v1/web-search/completions` must migrate to:

```
POST /api/v1/llm/completions
Body: { "selected_tools": ["web_search"], ... }
```

The dedicated web-search router is **removed** in Phase 2.

RAG Ingest

Phase 2 ingest endpoint (`POST /api/v1/rag/`) now accepts a `folder_id` form field (optional). Existing integrations not sending `folder_id` continue to work (documents remain unfolded).

Generated: 2026-07-01 | Repository: ai-enterprise-gpt-backend

